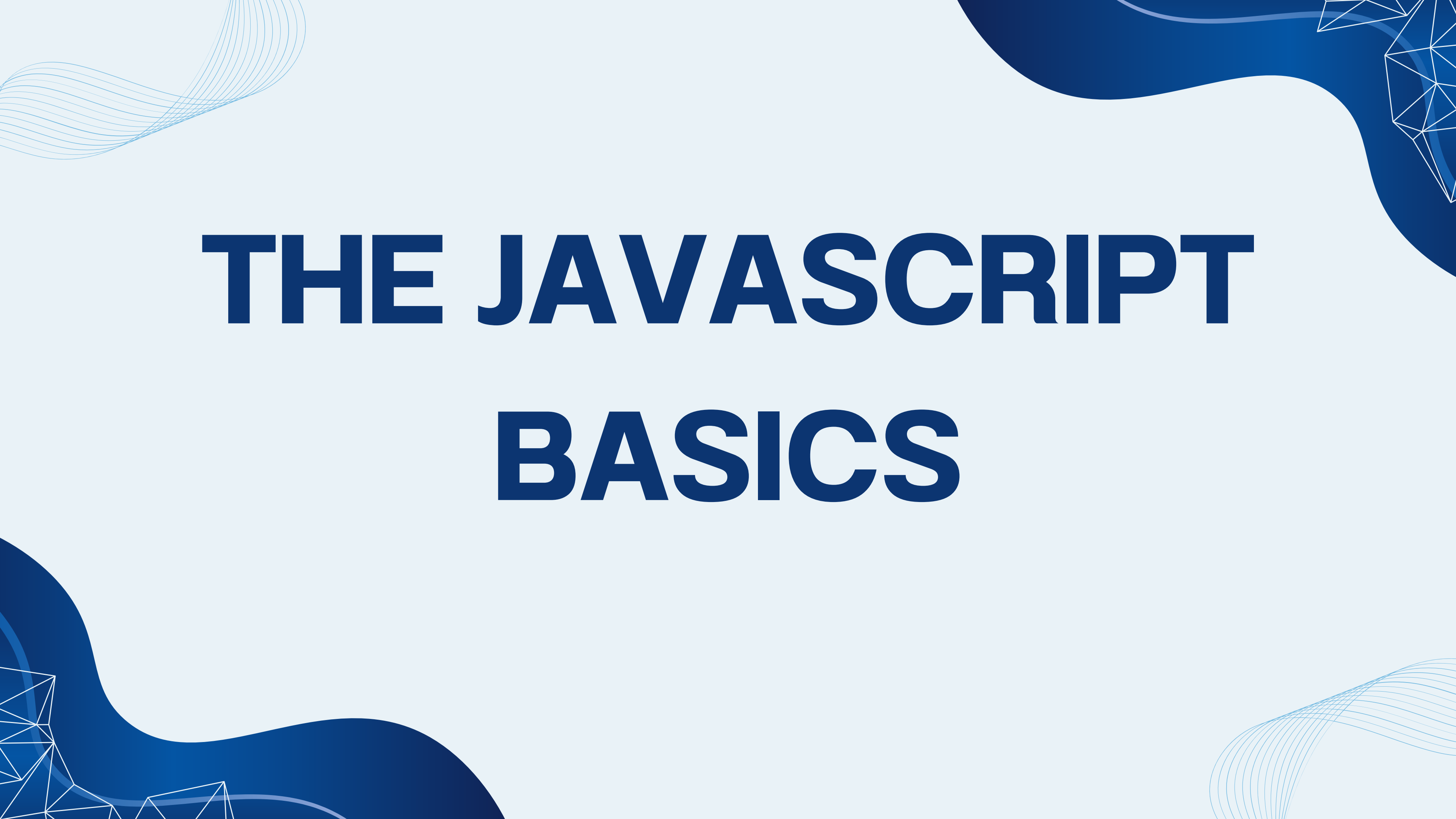




THE JAVASCRIPT BASICS

JAVASCRIPT BASICS (CAREY, VODNIK, 2022)

As HTML is assigned for the structure of the website and CSS is for styling it, JavaScript (JS) makes the website interactive and complex through the implementation of intricate features such as displaying timely content updates and determining what happens when users click, hover, or type within specific elements in the website.

It is a case-sensitive scripting language for webpages and is considered the third standard for web technologies after HTML and CSS. As each language serves a different purpose, it is necessary to use a separate file for each, but it must be in the same directory, which can be a folder on the computer. For JavaScript, a .js file extension is used.

This idea is called “separation of concerns,” meaning a program is separated into distinct sections. In context, the “program” refers to the website, while the “sections” refers to the separate files of HTML, CSS, and JavaScript. They might be separated, but they have a unique function needed for the website.

Just like in CSS, JavaScript can be referred to internally and externally, such as:

Internal:

```
<!DOCTYPE html>
<html>
  <head>
    <title>Internal</title>
    <script type="text/javascript">
      document.write("A sample script");
    </script>
  </head>
</html>
```

External:

```
<!DOCTYPE html>
<html>
  <head>
    <h2>External</h2>
    <script src="jsfilename.js">
    </script>
  </head>
</html>
```

- In **internal or embedded JavaScript**, the JS code is placed and can appear multiple times inside the `<script>` tags of an HTML document using `<script>` tags that encloses the JavaScript codes.
- While in **external JavaScript**, the separated .js file is linked using `<script>` tags with the `src` attribute are usually placed within the `<head>` tags. It applies the JavaScript code to the webpage so that it can have an effect on the HTML document and anything else on it.
- As practiced, all codes can be in one HTML code, but it will lead to repetitive sets of code that might confuse as the website grows. <!--EndFragment--> </body>
</html>

JAVASCRIPT COMMENT

- It is the line of code that browsers ignore and is used on a specific line or block of codes to add notes such as the program name and date or notes for future programmers that might modify the program.
- A **line comment** is used by adding two (2) slashes (//) before the note or comment, while **block comments** are used to hide multiple lines of codes by adding /* to the first line of the block and enclosed by a */ after the last character in the block

JAVASCRIPT STATEMENT

It is an individual line of code that is enclosed within the <script> tags.

```
<script>  
document.write("<p>Best choices</p>");  
</script>
```

This sample statement writes the text "Best choices" without quotation marks on a webpage. It uses the write() method of the Document object, and each statement must end with a semicolon (;).

JAVASCRIPT OBJECT

It is a programming code that can be treated as an individual unit or component. A JS object can be defined and created by programmers based on the program's need aside from available objects from the **Document Object Model (DOM)** and **Browser Object Model (BOM)**

```
bankLoan.calcPayments(60);
```

An object such as bankLoan is created to calculate the number of payments to pay off a bank loan. Statements such as this are often grouped into logical units called **procedures** that perform specific tasks.

Additionally, procedures associated with an object are called **methods** which are actions that can be performed on it. In this sample, calcPayments(), which is defined to calculate the number of payments to pay off the loan is the associated method.

Arguments are used and placed inside the parentheses beside it to provide more specific information about the method. In this case, (60) is the argument that specifies the number of months until the loan is paid off.

JAVASCRIPT OUTPUT METHODS

- Any text, graphics, and other information displayed on the webpage is part of the Document object, which is one of the most commonly used objects in DOM. The **write() method** of the Document object is used to write custom information such as username or address.
- Moreover, this also requires a **text string** as an argument, which is the text contained within double or single quotation marks. The combination of these concepts becomes `document.write("<p>Best choices</p>");`

JAVASCRIPT OUTPUT METHODS

- Any text, graphics, and other information displayed on the webpage is part of the Document object, which is one of the most commonly used objects in DOM. The **write() method** of the Document object is used to write custom information such as username or address.
- Moreover, this also requires a **text string** as an argument, which is the text contained within double or single quotation marks. The combination of these concepts becomes `document.write("<p>Best choices</p>");`

JAVASCRIPT OUTPUT METHODS

- **console.log** is a basic method used to write text to the console. A console is where messages are displayed that are invisible to users and are separated from the HTML/CSS environment. However, this can be visible in a browser's developer tool console.
- It is possible to input text in the argument with or without quotation marks and stack arguments separated by commas or + signs,
- The **alert()** method is used to display an alert box that has a message and an OK button. The text in the argument is the information users are forced to see, so it is better not to overuse this method

JAVASCRIPT OPERATOR

- **Assignment operators** allow assigning of values to variables and constants. It commonly uses the equal sign (=).
- **Arithmetic operators** express mathematical operations and accept numerical values and variables. It includes addition (+), subtraction (-), multiplication (*), division (/) remainder (%), power (**), and increment (++) that adds one (1) in the operand and decrement (--) that subtracts one (1).
- **Logical operators** work with Boolean-type values such as true or false. It includes a conjunction that performs logical AND (&&), an alternative that performs logical OR (||), and a negation that performs logical NOT (!).
- **Comparison operators** compare two (2) variables or operands. It includes not equal or equal to the value of the variable or operand (!=, ==), equal value and equal type (===), less than and greater than (<, >), and less than or equal to and greater than or equal to (<=, >=)

JAVASCRIPT VARIABLE

- **Assignment operators** allow assigning of values to variables and constants. It commonly uses the equal sign (=).
- **Arithmetic operators** express mathematical operations and accept numerical values and variables. It includes addition (+), subtraction (-), multiplication (*), division (/) remainder (%), power (**), and increment (++) that adds one (1) in the operand and decrement (--) that subtracts one (1).
- **Logical operators** work with Boolean-type values such as true or false. It includes a conjunction that performs logical AND (&&), an alternative that performs logical OR (||), and a negation that performs logical NOT (!).
- **Comparison operators** compare two (2) variables or operands. It includes not equal or equal to the value of the variable or operand (!=, ==), equal value and equal type (===), less than and greater than (<, >), and less than or equal to and greater than or equal to (<=, >=)

JAVASCRIPT VARIABLE

In JavaScript, five (5) primitive data types specify which type of value a variable contains.

- **String** – represents textual data.
- **Boolean** contains a logical entity that may return a TRUE or FALSE value.
- **Number** – any digit that is either an integer or a decimal.
- **Null** – an unknown or missing value.
- **Undefined** – refers to a variable with no value. The **undefined** keyword can be set in the variable.

DECLARING AND INITIALIZING VARIABLES

Before a variable is used, it must be declared first so it can store data and objects or initialized, which assigns it an initial value. Variables are declared and initialized using the **let** or **var** keywords.

CONTROL FLOW

Control flow statements, or conditional execution, break up the execution flow using decision-making, branching, and looping to enable programs to conditionally execute specific code blocks.

- **if statement** - It is the simplest control flow instruction in JavaScript. It checks a given condition and, depending on its Boolean value, either executes a code block or skips it. The code block should be separated using curly brackets
- The **else** keyword - is an optional part of the if statement, and it allows the addition of a second code block that will be executed only when the initial condition is NOT met.

LOOPS AND ITERATIONS

Loops are the second form of control flow statement. While control flow statements can change code behavior, loops are an easy way to repeat any fragment of the code as many times as possible, or until a stopping condition is met

- In a **while loop**, a code block is executed while a given condition is true but stops once the condition is false. When a code block is repeated, it is referred to as an **iteration**.
- An **infinite loop** is avoided by including at least one statement that eventually results in a false value for the condition. A code block uses a **counter**, whose value changes with each iteration, and once that counter fails to match the condition, the loop ends.
- **for loop** In the parentheses after the word for, there will be no single condition this time, unlike in the while loop. Here, it is divided into three fields by semicolons, and each field is assigned a different meaning



THANK YOU